بِسْمِ اللَّهِ الرَّحْمَٰنِ الرَّحِيمِ

In the name of ALLAH, the Beneficent, the Merciful

7

Distributed File System (DFS)

Compiled By

Dr Muhammad Sajid Qureshi

Contents

- Computer File System
  - Definition, goals, properties and modules
- Distributed File System
  - DFS architecture and flow model
  - Client-Server DFS
  - Sun NFS
- Storage Area Network (SAN)
- Challenges in DFS
  - File Name Resolution
  - Cache Consistency
  - File Writing Policy
  - File System Scalability

Content of this lecture is based on Chapter 12 of the book "Distributed Systems Concepts and Design" (5th edition), by George-Coulouris.

Computer File System

- A File System is used to control how data is stored to and retrieved from the storage media.
  - Without a file system, information placed in a storage area would be one large body of data with no way to tell where one piece of information stops and the next begins.
  - By separating the data into individual pieces or files, and giving each piece a name, the information is easily separated and identified.
    - A file is a logical organization of data

- There are various kinds of file systems.
  - Each one has different structure, logic, properties of speed, flexibility, security, size and more.
  - Some file systems have been designed to be used for specific applications.

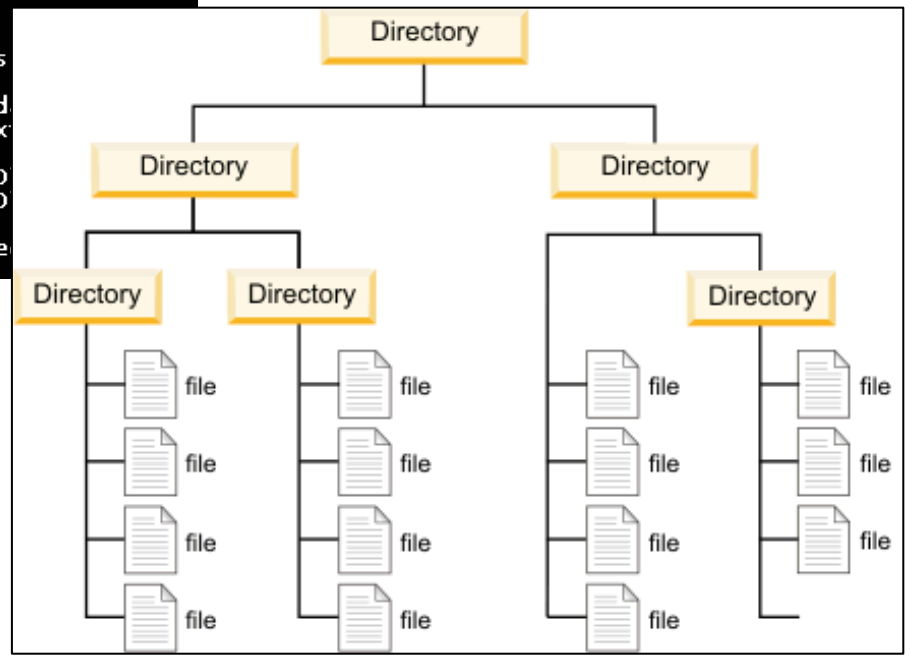
Computer File System

```
C:\Temp> dir
 Volume in drive C is C
 Volume Serial Number is 74F5-B93C

 Directory of C:\Temp

2009-08-25  11:59    <DIR>          .
2009-08-25  11:59    <DIR>          ..
2007-03-01  11:37         2,321,600 AdobeUpdater12345.exe
2009-04-03  10:01            27,988 dd_depcheckdotnetfx30.txt
2009-04-03  10:01               764 dd_dotnetfx3error.txt
2009-04-03  10:01            32,572 dd_dotnetfx3install.txt
2009-06-09  13:46            35,145 GenProfile.log
2009-08-05  12:11               155 KB969856.log
2009-04-20  08:37               402 MSI29e0b.LOG
2009-04-09  16:34            38,895 offcln11.log
2009-04-03  16:02    <DIR>          OfficePatches
2009-07-14  14:30    <DIR>          OHotfix
2009-08-25  10:52            16,384 Perflib_Perfd
2009-04-03  10:01             1,744 uxeventlog.tx
2009-08-25  11:42        50,245,632 WFV2F.tmp
2009-04-20  10:07             1,397 {AC76BA86-7AD
2009-04-20  10:13               617 {AC76BA86-7AD
              13 File(s)     52,723,295 bytes
               4 Dir(s)  83,570,208,768 bytes fre
```

Command Line Interface



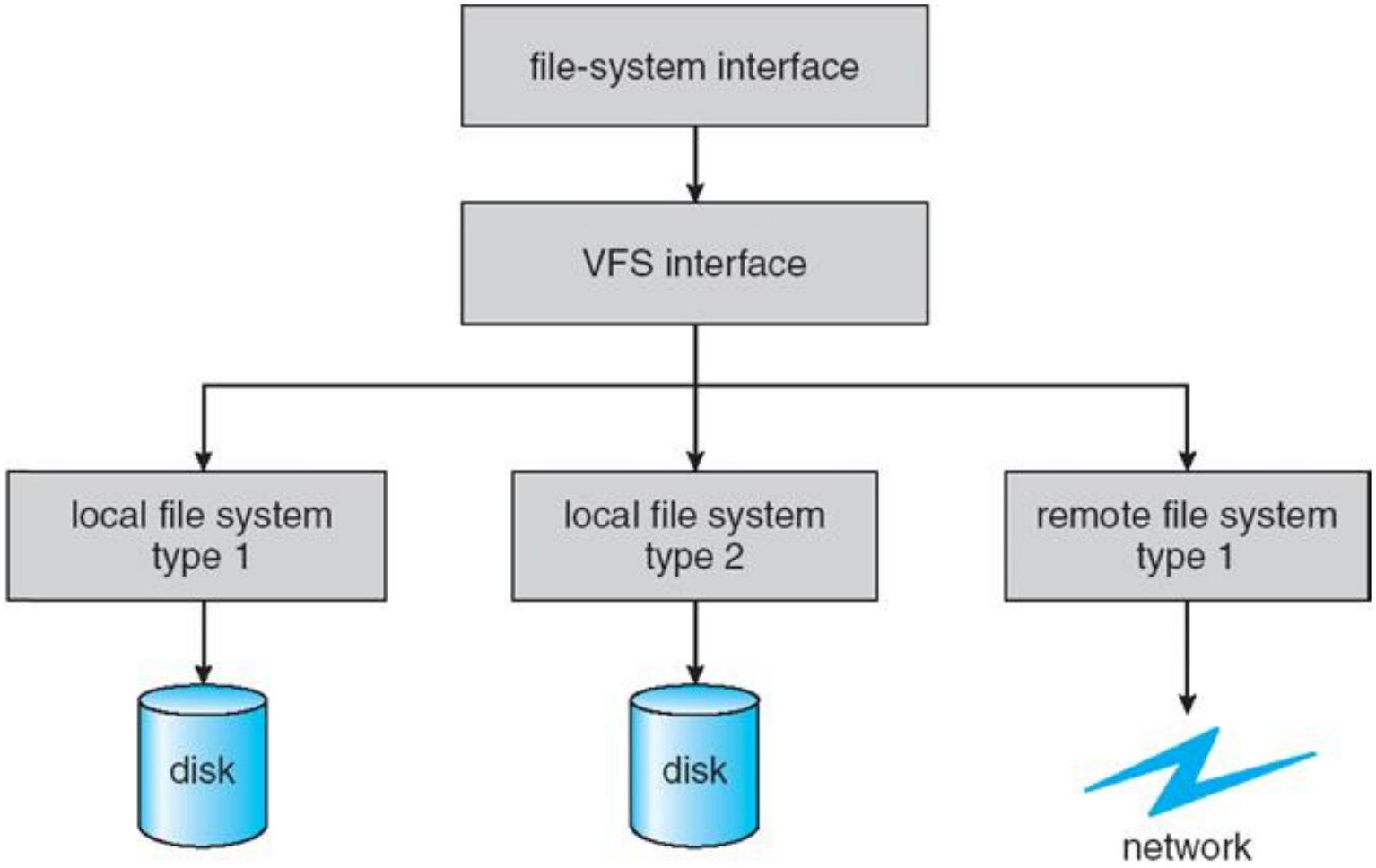
Graphical Interface

Computer File System

- File systems can be used on many kinds of storage devices.
  - It manages access to the content of files and their metadata.
  - It is responsible for:
    - Arranging storage space
    - Reliability
    - Efficiency
    - Tuning regarding the physical storage medium are important design considerations.

- Virtual File Systems (VFS) provide file access via a network protocol.
  - VFS is a mapping into a different file system used as a backing store.
  - VFS supplies the "files" on request by the clients

Computer File System



Properties of Storage Systems

	Sharing	Persistence	Distributed cache	Consistency	Example
Main memory	✗	✗	✗	1	**RAM**
File system	✗	✓	✗	1	**UNIX file system**
Distributed file system	✓	✓	✓	✓	**Sun NFS**
Web	✓	✓	✓	✗	**Web server**
Distributed shared memory	✓	✗	✓	✓	**IVY**
Remote objects (RMI/ORB)	✓	✗	✗	1	**CORBA**
Persistent object store	✓	✓	✗	1	**CORBA Persistent Object Service**
Peer-to-peer storage system	✓	✓	✓	2	**OceanStore**

Types of consistency:

1: Strict one-copy with slightly weaker guarantees.

2: Considerably weaker guarantees.

Modules of a File System

Directory module:	relates file names to file IDs
File module:	relates file IDs to particular files
Access control module:	checks permission for operation requested
File access module:	reads or writes file data or attributes
Block module:	accesses and allocates disk blocks
Device module:	performs disk I/O and buffering

File Attributes

File length
Creation timestamp
Read timestamp
Write timestamp
Attribute timestamp
Reference count
Owner
File type
Access control list

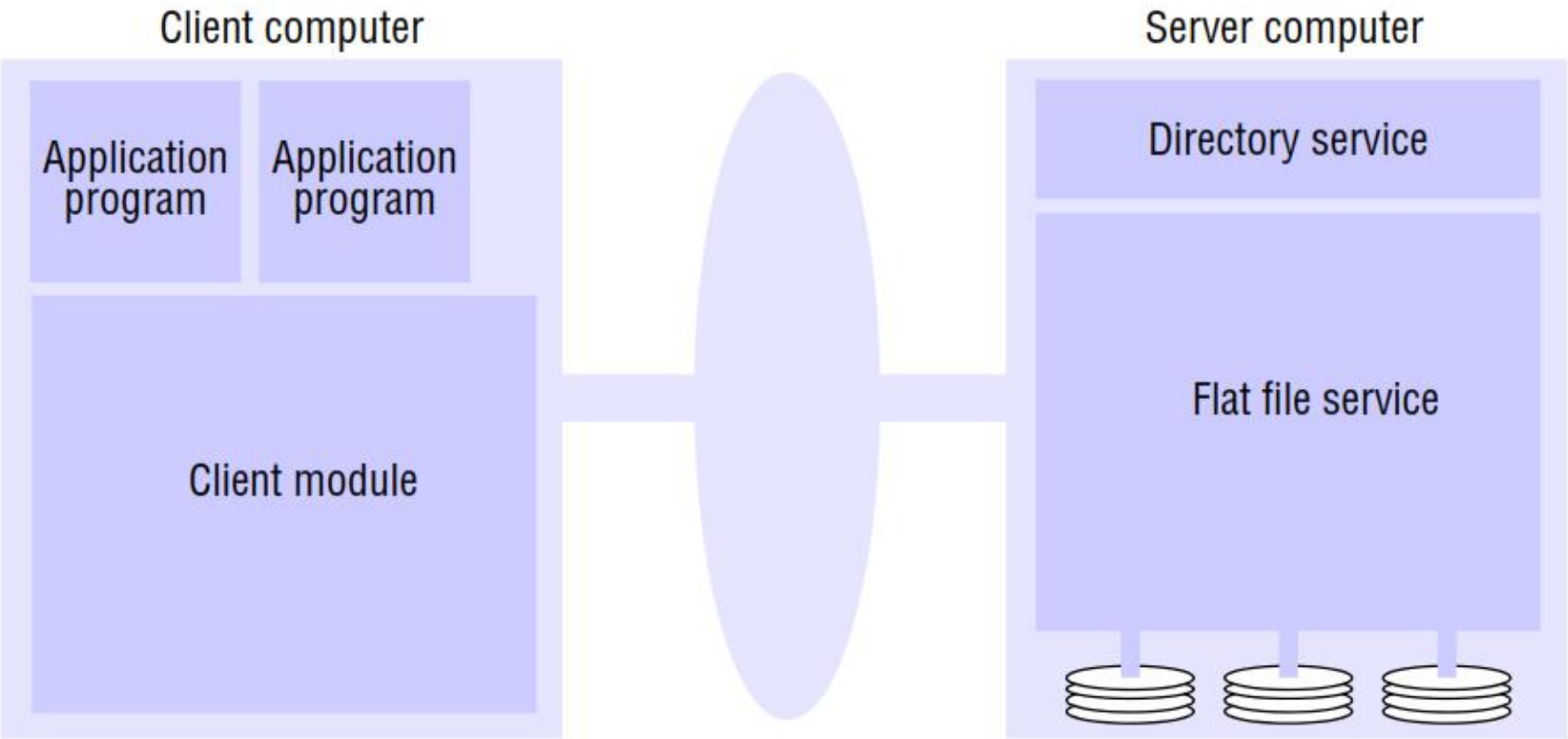
Distributed File System (DFS)

- Distributed File System
  - DFS is a file system that is distributed across a network.
  - It Implements a common file system that can be shared by all autonomous computers in a distributed system
    - For example, the Virtual File System (VFS) follows a Client-Server Architecture.
      - Microsoft's DFS
  - DFS Goals
    - Network transparency AND High availability
  - Architectural options
    - Fully distributed: files distributed to all sites (participants)
      - Issues: performance, implementation complexity
    - Client-server Model
      - Fileserver: dedicated sites to store files. It also performs the storage and retrieval operations
      - Client: rest of the sites that use the servers to access files
      - Issues: Single Point of Failure, Scalability

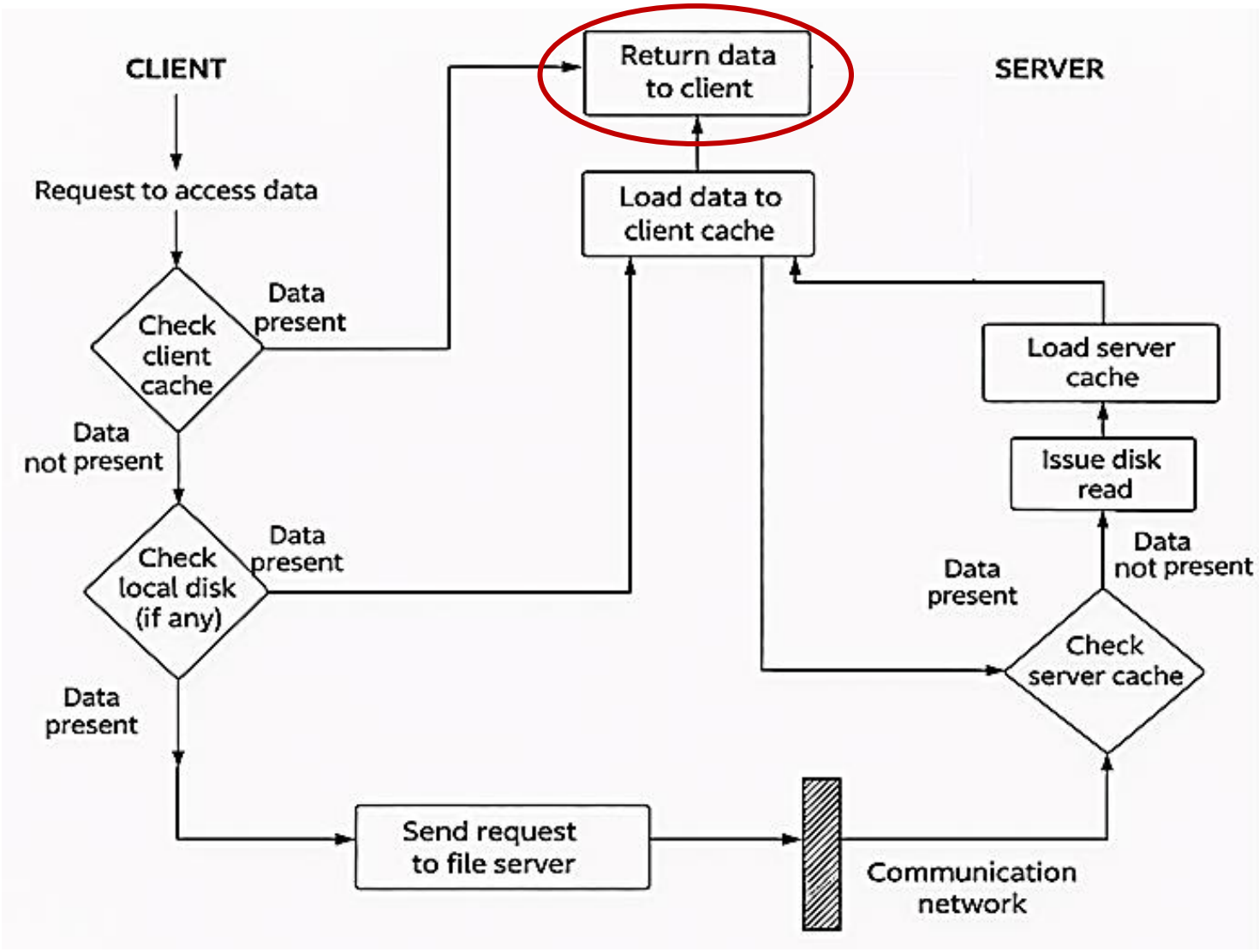
DFS Requirements

- Requirements of a DFS
  - Transparency (Access and Location)
  - Performance
  - Scalability
  - Concurrency Control
  - Fault Tolerance
  - Security

DFS Architecture



Flow Model of a DFS



Sun NFS

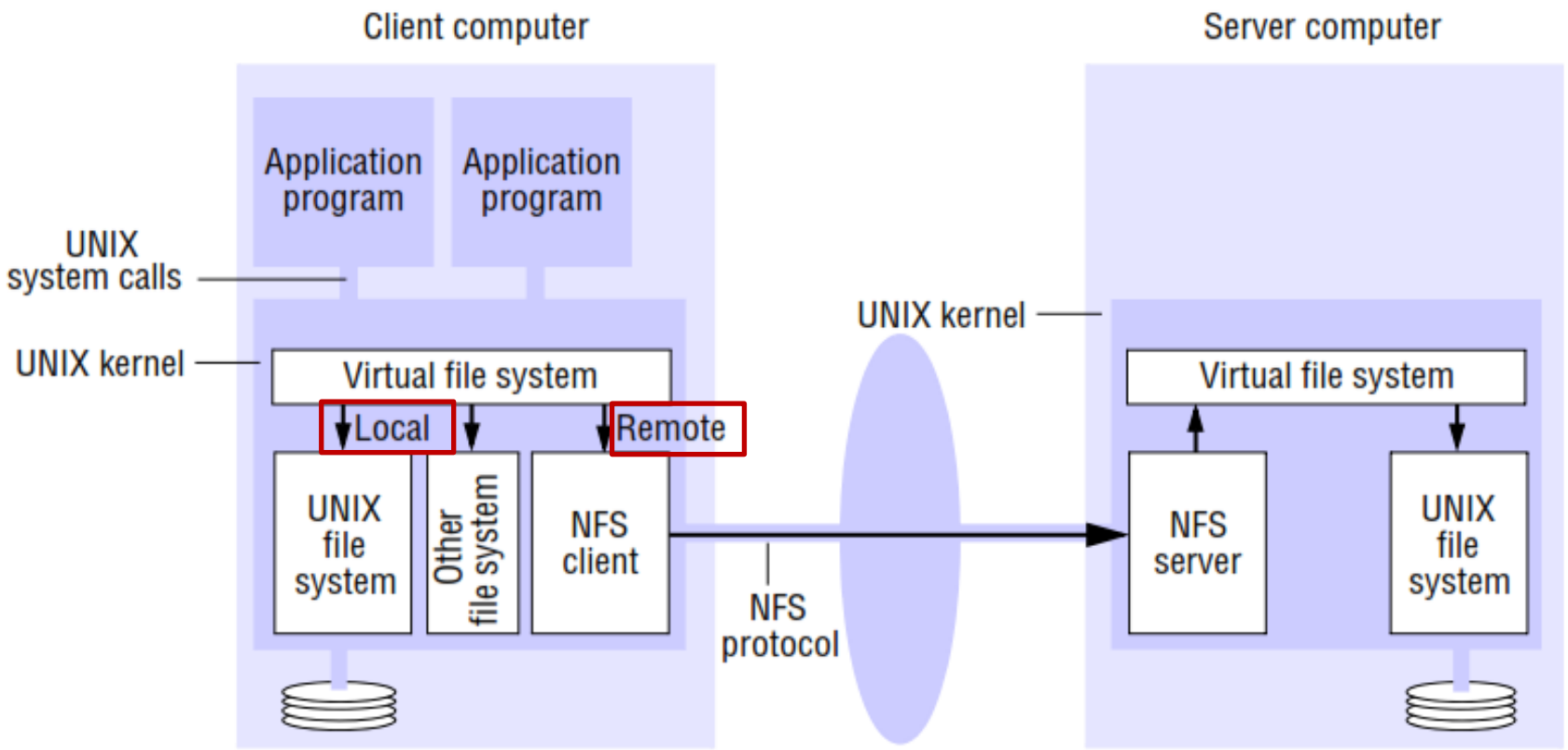
- Network File System (NFS)
  - NFS is a distributed file system protocol originally developed by Sun Microsystems in 1984.
  - It allows a user on a client computer to access files over a network much like local storage is accessed.
    - NFS, like many other protocols, uses the RPC system.
  - The Network File System follows an open standard that is defined in RFCs.
    - Anyone can implement the protocol.

Virtual File System

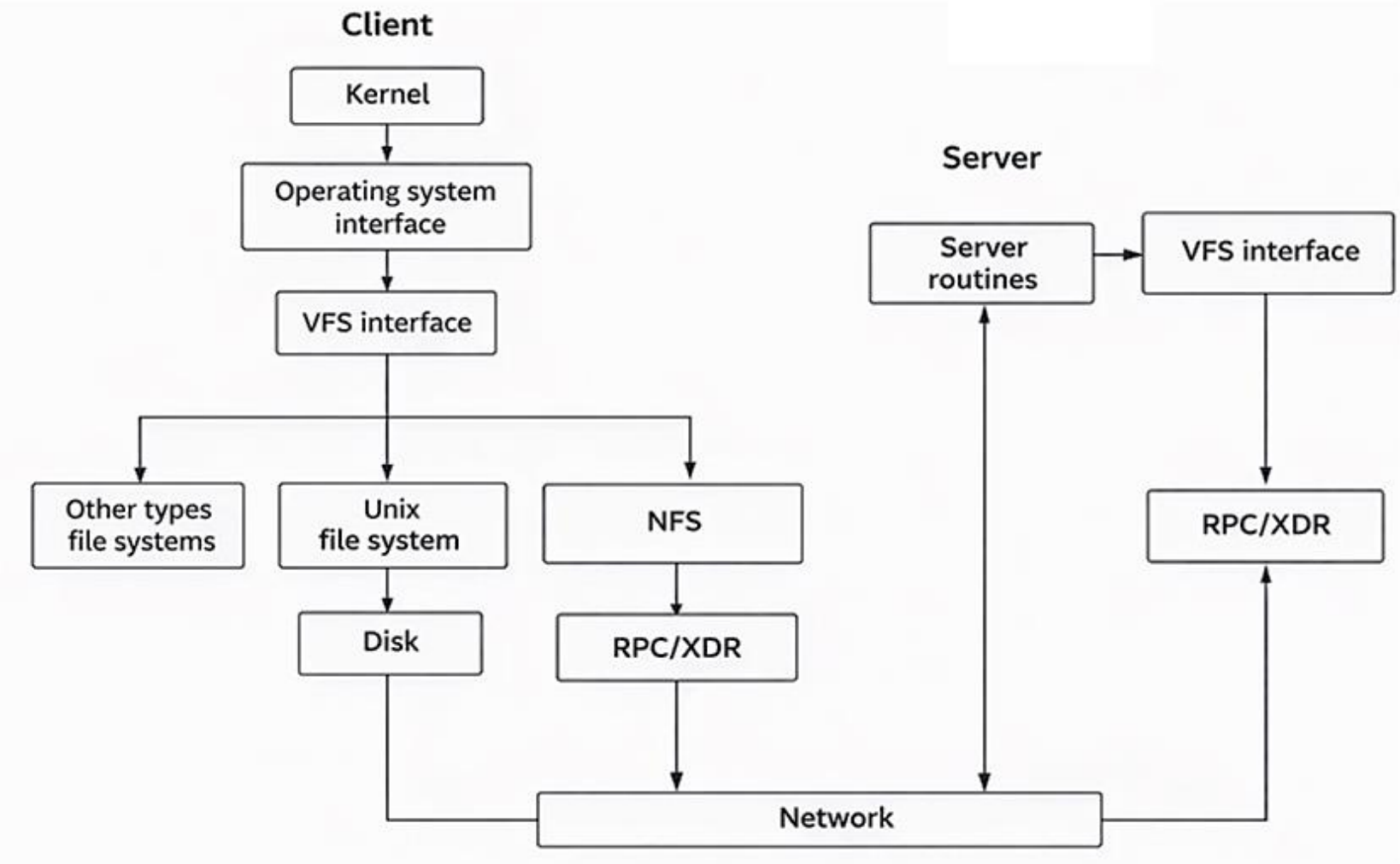
Virtual File System (VFS)

- VFS is a file system interface that allows NFS to support different file systems.
- Requests for operation on remote files are routed by VFS to NFS.
- Requests are sent to the VFS on the remote using the:
  - Remote Procedure Call (RPC) AND
  - External Data Representation (XDR)
- VFS on the remote server initiates files system operation locally
- Vnode (Virtual Node)
  - There is a network-wide V-node for every object in the file system (file or directory)- equivalent of UNIX I-node
  - V-node has a mount table, allowing any node to be a mount node

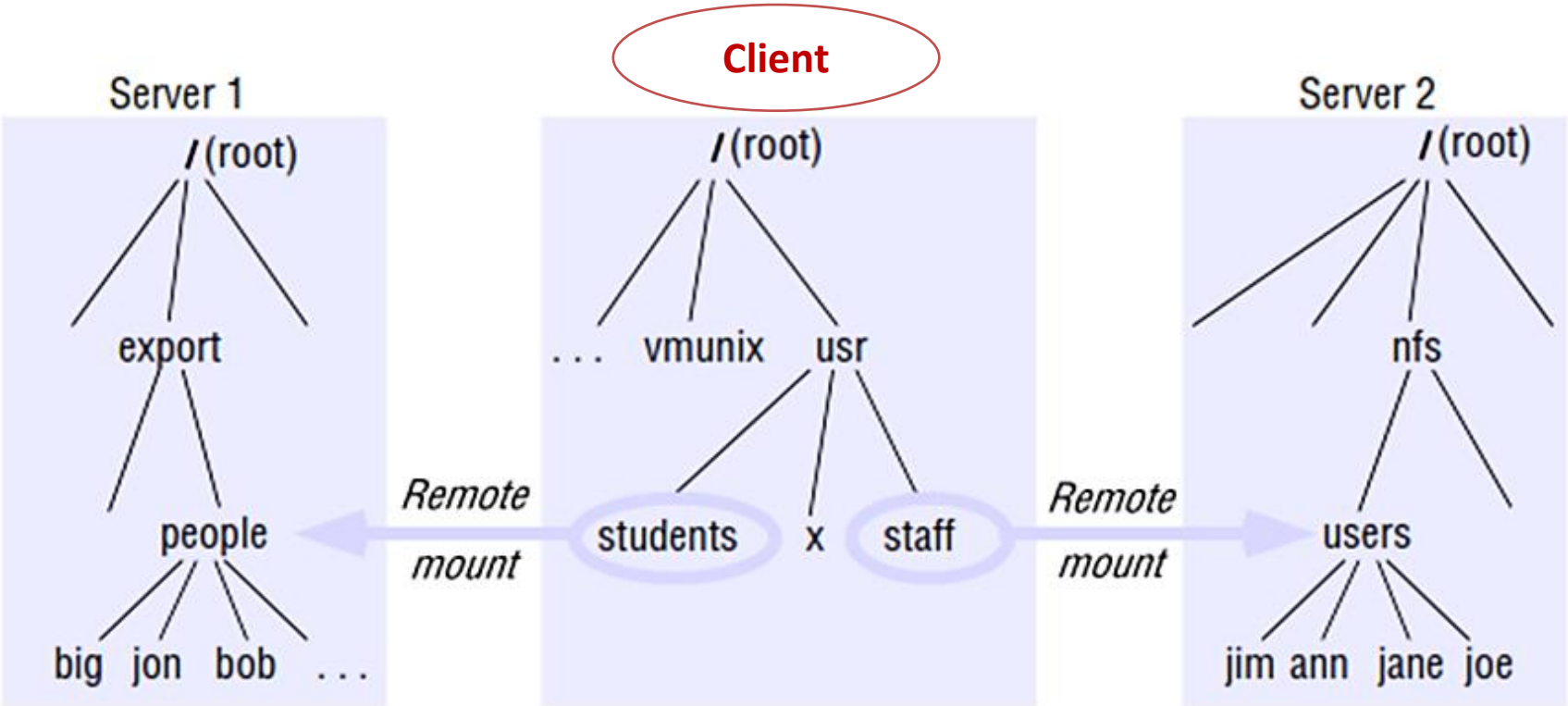
DFS Architecture



Sun NFS - Architecture



Sun NFS - Architecture

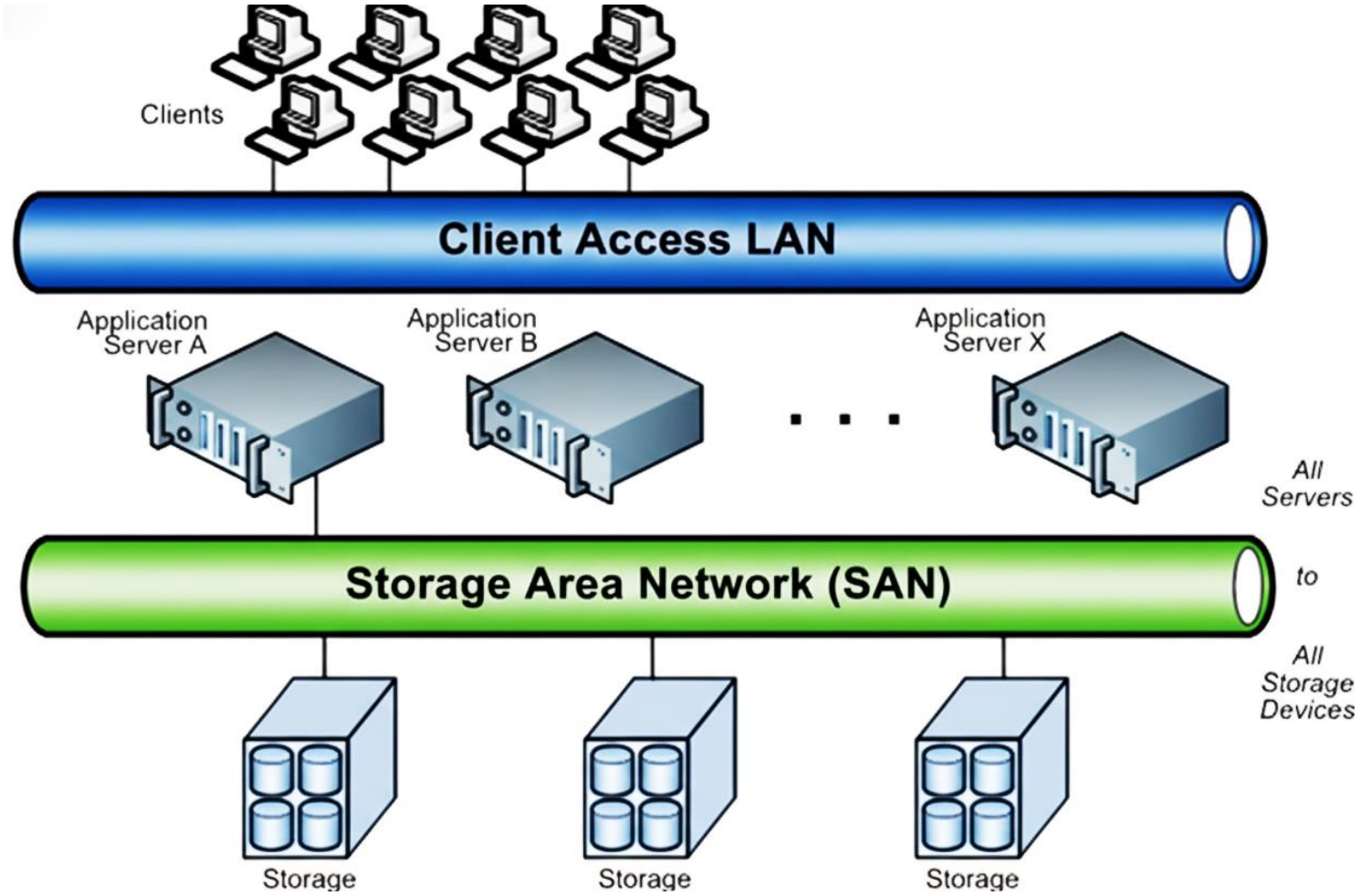


The file system mounted at */usr/students* in the client is actually the subtree located at */export/people* in Server 1; the filesystem mounted at */usr/staff* in the client is actually the subtree located at /nfs/users in Server 2.

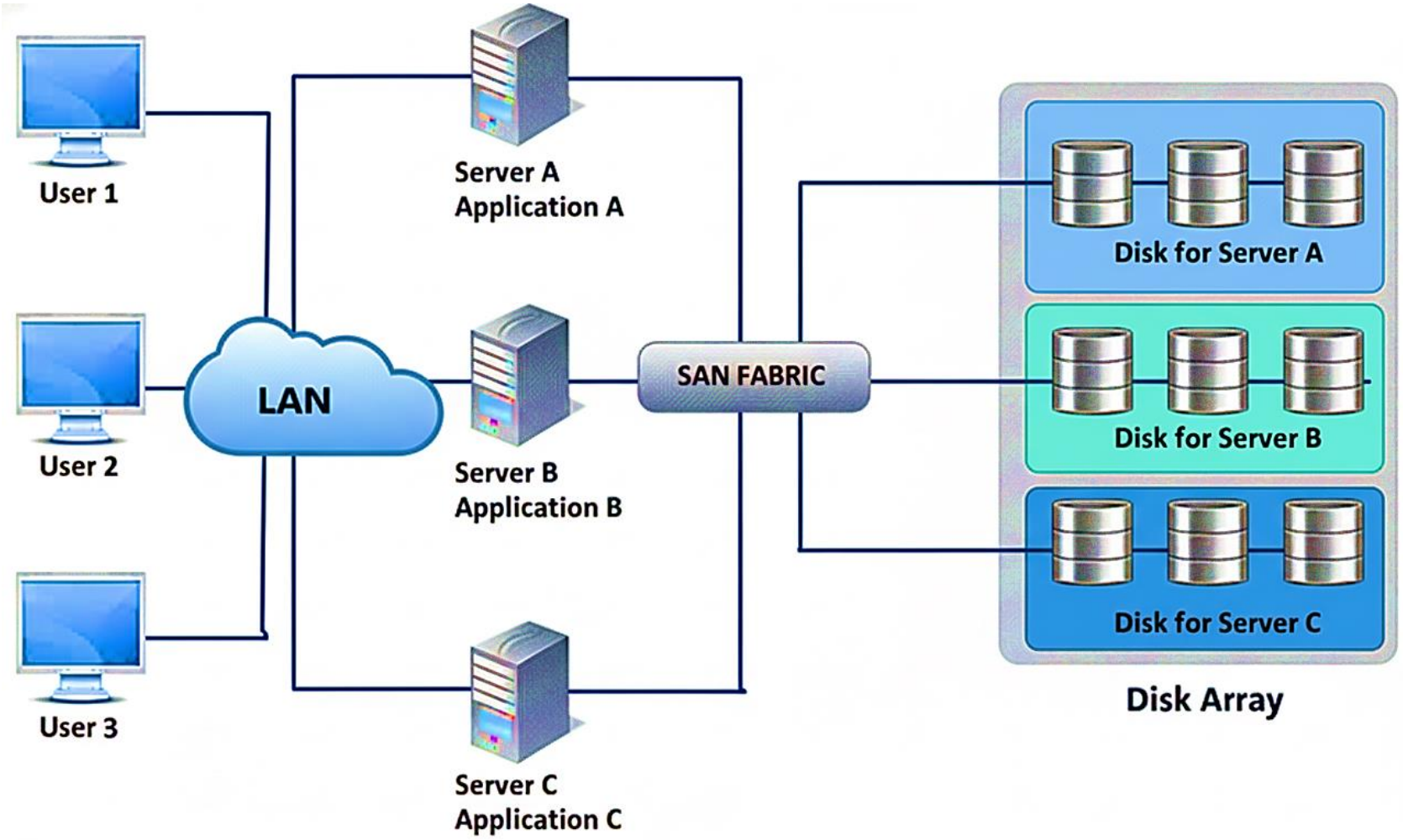
Storage Area Network (SAN)

- Storage Area Network (SAN)
  - A SAN is a network that provides access to consolidated, block level data storage.
    - SANs are primarily used to enhance storage devices, such as Disk Arrays, Tape Libraries, and Optical Jukeboxes, accessible to servers.
    - The devices appear to the operating system as locally attached devices.
  - A SAN typically has its own network of storage devices.
    - The devices are generally not accessible through the Local Area Network (LAN).
  - A SAN does not provide file abstraction, only block-level operations.
    - However, file systems built on top of SANs do provide file-level access and are known as shared-disk file systems.

Storage Area Network



Storage Area Network (SAN)



Challenges in DFS

Design Issues in DFS

- Unique Name: each object in a file system (file, directory) has a unique name
- Name resolution: mapping a name to an object or multiple objects (replication)
- Name space: collection of names with or without name resolution mechanism
  - Concatenate name of host to names of files on that host
  - Advantage
    - Unique filenames, simple resolution
  - Disadvantages
    - Moving file to another host requires changing its name and the applications using it
    - Conflicts with network transparency
- Mounting remote directories onto local directories
  - Requires that host of the remote directory is known
  - After mounting, files referenced are location-transparent
    - File name does not reveal its location
- Have a single global directory - All files belong to a single name space
  - Limitation: having unique system wide filenames require a single computing facility or cooperating facilities

Challenges in DFS

- Major challenges in design of the DFS
  - File Name Resolution
  - Cache Consistency
  - File Writing Policy
  - File System Scalability

Name Resolution

- Adding contexts to the namespaces
  - Solve the problem of system-wide unique names, by partitioning a name space into contexts (geographical, organizational, etc.)
  - Name resolution is done within that context
  - Interpretation may lead to another context
    - File Name = Context + Name local to context
- Using the Naming Server (The DNS Approach)
  - Process that maps file names to objects (files, directories)
  - Implementation options
    - Single name Server
      - Simple implementation, reliability and performance issues
    - Several Name Servers (on different hosts)
      - Each server responsible for a domain.
        - » Example: Client requests access to file 'A/B/C'
      - Local name server looks up a table (in kernel)
      - Local name server points to a remote server for '/B/C' mapping

Cache Consistency

- How to keep caches of clients and server consistent?
  - Server initiated
    - Server informs cache managers when data in client caches is out-of-date.
    - Client cache managers invalidate out-of-date data or retrieve new data
  - Client initiated
    - Cache managers at the clients validate data with server before returning it to clients
  - Both the approaches involve extensive communication
- Concurrency Control
  - Prohibit file caching when concurrent-writing when several clients open a file, at least one of them for writing
    - Server informs all clients to purge (remove) that cached file
  - Lock files when concurrent-write sharing (at least one client opens for write)

File Writing Policy

- File writing policy
  - It guides when should the modified cache be sent to the server, once a client writes into a file (and the local cache)
- Possible Options:
  - Write-through: all writes at the clients, immediately transferred to the servers
    - Advantage: reliability
    - Disadvantage: low performance, as it does not take advantage of the cache
  - Delayed writing: delay transfer to servers
    - Advantages:
      - Many writes take place (including intermediate results) before a transfer
      - Some data may be deleted
    - Disadvantage: Low reliability
  - Delayed writing until file is closed at client
    - For short open intervals, same as delayed writing
    - For long intervals, reliability problems

File System Scalability

- Scalability means that the design support a growing system.
  - In server-initiated cache invalidation complexity and load grow with size of system.
  - Possible solutions:
    - Do not provide cache invalidation service for read-only files.
    - Provide design to allow users to share cached data
- Design options for scalability of a file server:
  - Multithread Server processes
  - Symmetric Multiprocessors (SMPs)
  - Clusters
- Expected semantics: a read will return data stored by the latest write
  - All read and writes go through the server
    - Disadvantage: communication overhead
  - Use of lock mechanism
    - Disadvantage: Locked file is not always available to all its users

Content's Review

- Computer File System
  - Definition, goals, properties and modules
- Distributed File System
  - DFS architecture and flow model
  - Client-Server DFS
  - Sun NFS
- Storage Area Network (SAN)
- Challenges in DFS
  - File Name Resolution
  - Cache Consistency
  - File Writing Policy
  - File System Scalability

You are Welcome !

Questions ?

Comments !

Suggestions !!